

29/01/24

PYTHON

Manday.

* What is Python?

Python is a object oriented programming language like as a Java, it's called an interpreted language. The python is developed by the Nethexland scientist Gudio van russam in the year of 1989. Before that he worked on different programming languages like ABC programming, Modular programming also. These two programming based on that he started the python programming.

Some applications of Python :-

- 1) Game development
- 2) Artificial intelligence
- 3) Machine learning
- 4) Web development
- 5) Software sector
- 6) Finance sector like, censex

Features of Python :-

- Easy to learn
- Secured
- It is an interpreted language
- Simple syntax
- Debugging language
- It is open source

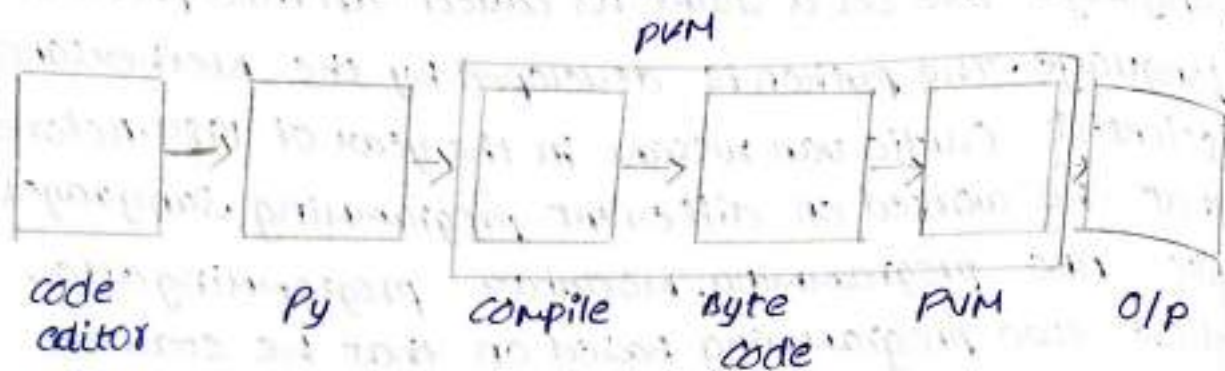
The working of python

The working of Python:-

- for writing into python code in code editor by saving in .py
- This .py code is human readable format that is reason the .py file is compile, after compile the human readable format is converted into Bytecode.

that byte code is machine understandable language, there after the output is from through python virtual image.

Internal Working of Python:-



The difference b/w the compiler and interpreter.

Compiler	Interpreter
1) Compiler is very faster as conversion occurs before the execution	1) The interpreter runs slower as the execution occurs simultaneously.
2) It requires more memory to translate the source code	2) It requires less memory
3) In compiler the debugging is more complex.	3) In interpreter debugging is very easier by line-to-line execution of code.

versions of Python programming language:-

- The first version released in 1989 - 0.9.0
- Version 1.0 in 1994.
- Version 2.0 in 2000.
- Version 3.0 in 2008.
- The latest version of Python is 3.12.1 (B-version) - 2023
- The current version 3.13 - (B-version).

Basic syntax :-

→ print("WELCOME TO KLU")

WELCOME TO KLU.

```
→ print('Welcome to KLU')
```

Welcome to KLU

→ print("KLU")

'KLU'

```
→ print(" " "KLU" " ")
```

KLU

→ print('print("KLU")')

```
"kLU" print("kLU")
```



30/04/23

The python has two basic modes. They are:-

- 1) Interactive Mode

- 2) scripting mode

1) Interactive Mode :-

1) Interactive Mode :-
Interactive mode is a command line shell which gives immediate results for each statement, while running previously output statement in active memory. The interactive mode is a good way to play around the various on syntaxes.

Ex:- >>> 3<-1

 $\gg 3$

```
>>> print(3*4)
```

22 12

```
>>> print('helloworld!')
```

>>> Hello world!

333.11 KLU 11*2

222 'KLUKLU'

》》》 "KLO " * 2

222 'KLU KLO'

2) scripting mode:-

The scripting mode is the mode where the scripted and finished .py file are run in the python interpreter (line-by-line)

```
Ex:- a=5
      b=7
      print(a*b)    => 35
```

Python identifiers and keywords:-

- Python identifiers working as a variable.
- An identifier begins with letters (A to Z or a to z) or an underscore also.
- Python does not permit the use of special character like @, #, %, !, &
- Python identifiers start with lower case or upper case.
- Identifiers cannot be a keyword.
- Identifiers are case sensitive
- White spaces are not acceptable in identifiers.

Ex:- >>> var = 0

>>> print(var)]

Syntax error: unmatched ']'

>>> print(var)

2

>>> -bac = 34

>>> 123abc = 35

error: invalid decimal literal

>>> in = 45

Error: invalid syntax

>>> abc @ = 78

Error: invalid syntax

>>> abx123 = 87

Accepted

NOT accepted

var = 0

-bac = 34

abc123 = 87

123abc = 35

abc@ = 78

keywords (35)

- 1) In Python keywords are also called as a reserved keywords.
- 2) In Python total (35) keywords are available
- 3) These reserved keywords are not used in variables or identifiers.

false	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	important	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	non local	yield
break	for	not	

Indentation and comments:-

- 1) Comments in a python we are using the # sign that is not inside a string literals and begins a comment.
- 2) All characters after # and upto the physical line end. Or part of the comment, and the python interpreter ignores them.

Types of comments in python:-

- > Single line comments
- > Multi line comments
- > String literals
- > String comments

Why you are using comments in python?

- > Enhance the code readability
- > Explaining code to others.
- > Understanding code if studied after some time.
- > Sharing code with the software developer
- > Collaborating with multiple people

Example for single line comment:-

#python code

a=5

b=7

print(a+b)

Example for multiple line comment:-

#python code

a=5

b=7

#c=8

print(a+b)

String literals:-

Python ignores the string literals that are not assigned to a variables so we can use the string literals as a python comments.

String literals single line comments:-

'python programming'

String literals multiple line comments:-

"""python programming"""

Note:-

a = 'abc123' - string

b = """python programming""" - string

Doc string comments:-

These are string literals with triple quotes that are appeared right after the function

Advantages of comments in python:-

1) Code readability

2) To explain the code to project manager they can easily understand.

3) Prevent the execution of the code is easily controlled.

Indentation in Python:-

python indentation refers to adding white spaces before a statement to a particular block of a code.

Syntax:-

C

```
if (condition)
```

```
{
```

```
    Statement,
```

```
}
```

```
else
```

```
{
```

```
    Statement,
```

```
}
```

python

```
if (condition):
```

Indentation → statement;

```
else:
```

Indentation → statement

Implicit and explicit of line joining method:

Implicit:-

In it the no. of open codes should be equal to the no. of closed codes.

```
print("Welcome
```

syntax error:- in complete input

```
print("Welcome"
```

```
    "to KLU")
```

```
Welcome to KLU
```

```
print("Welcome to KLU")
```

```
Welcome
```

```
to
```

```
KLU
```

Explicit:-

In it we will use the \

```
(print("Welcome \ print("Welcome \
```

```
to KLU") to KLU")
```

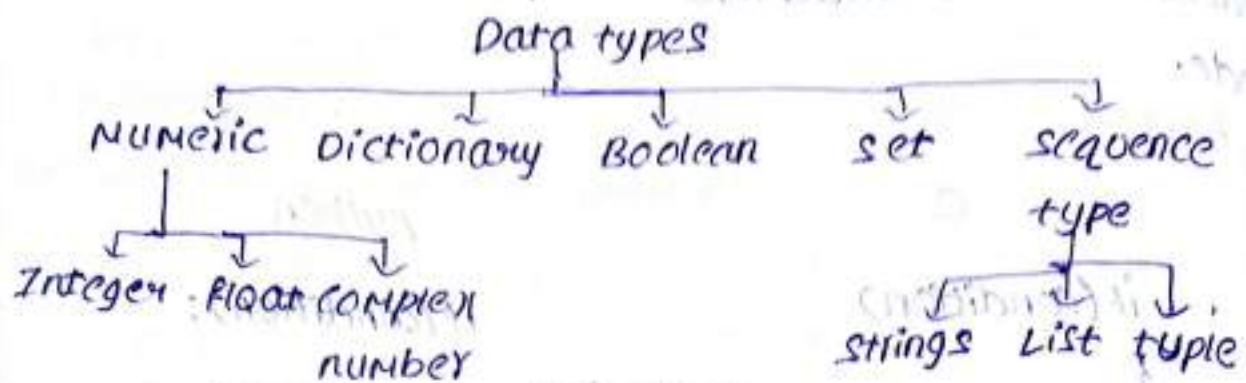
```
Welcome to KLU
```

```
or, Welcome to KLU.
```

pr.

8/8/24

Data types :-



→ Data types represents the kind of value that tells what operations can be performed on a particular data.

Numeric:-

In the Numeric type which divided into 3 types.

- 1) Integer
- 2) Float
- 3) Complex numbers

Python type() :- The type() is a function either it returns the type of object, it returns the type of new type of object.

→ It is performed based on the object pass.

Syntax:-

type(object)

Integer:-

It returns the value without decimal

Ex:-

a=10

print(type(a))

integer

10

a=10

b=float(a)

print(b)

print(type(b))

float :- It returns the value with decimal number.

EX :-

```
a=10.50
```

```
print(type(a))
```

```
print(a)
```

```
float
```

```
10.50
```

```
a=10.50
```

```
b=int(a)
```

```
print(b)
```

```
print(type(b))
```

```
10.50
```

Complex Numbers :- It is a combination of both real and imaginary part

EX :- $a = 10 + 3i$

```
print(type(a))
```

Operators :-

Python operators :- The operator is generally used to perform some operations on values and variables.

-> The operators in b/w operands use as it can perform some actions.

Operators : +, -, /, *

Operands : values

Types of Operators :-

1) Arithmetic operators

2) Comparison operators

3) Logical operators

4) Bit-wise operators

5) Assignment operators

6) Membership and identity operators

1) Arithmetic operators :- It can perform mathematical operations like addition, subtraction, multiplication, floor division.

EX :- $a = 10$

$b = 20$

```
print(a+b)
```

print(a+b);

print(a*b)

print(a*b)

print(a/b)

print(a||b)

Floor division:- The floor division operation that returns the largest integer i.e less than or equal to the result of the division. We can perform the floor division by using two forward slashes.

Syntax:-

a = dividend // divisor

2) comparison operators:- It returns the true or false accordingly to the

EX:-

a=10

b=20

print(a>b) $\Rightarrow$ # 10>20 = 0

print(a<b) # 10<20 = 1

print(a==b) # 10==20 = 0

print(a!=b) # 10!=20 = 1

print(a>=b) # 10>=20 = 0

print(a<=b) # 10<=20 = 1

3) Logical operators:- It performs the logical-AND, logical-OR, logical-NOT operations.

logical-AND = and

logical-OR = or

logical-NOT = not

Truth table:-

	and	or	not
T-T	T	T	F
TF	F	T	T
FT	F	T	T
FF	F	F	F

EX:-

a = true

b = false

print(a and b)

print(a or b)

print(not a)

print(not b)

4) Bitwise operators:- We are using bit-wise operators to operate on binary numbers. It can following some operators.

1) bit-wise AND (&)

2) bit-wise OR (|)

3) bit-wise NOT (~)

4) bit-wise XOR (^)

5) bit-wise left shift (>>)

6) bit-wise right shift (<<)

EX:-

a = 10

b = 4

print(a & b)

print(a | b)

print(a ^ b)

print(a ~ b)

print(a >> b)

print(a << b)

5) Assignment operators:- We can assign the values to the variables.

=> =

EX:- a = 10

b = 20 (equal to =)

a = a + b

a += b => a = a + b (short end operator)

a -= b => a = a - b

a /= b => a = a / b

a *= b => a = a * b

a //= b => a = a // b

a **= b => a = a ** b

$a**b \Rightarrow a = a**b$

$a\&b \Rightarrow a = a\&b$

$a|b \Rightarrow a = a|b$

$a\wedge b \Rightarrow a = a\wedge b$

$a>>b \Rightarrow a = a>>b$

$a<<b \Rightarrow a = a<<b$

Program:-

`a=10`

`b=20`

`b+=a`

`b=a+b`

`print(b)`

6) Identity operator:-

-> It checks the comparison of variables in python

-> They are used to check if two (variables) values are located on same memory location

-> Two variables that are equal do not imply that they are identical.

-> We are using two operators.

1) `is`

2) `is not`

`is` - It is used if condition true both operands are identical.

`is not` - If true both operands are not identical.

Ex:- `a=10`

`b=20`

`c=a`

`print(a is not b) - true`

`print(a is c) - true`

Membership operators:-

The Membership operator that are used to test whether a value or variable in sequence. We are using two operators.

1) in

2) not in

in - returns true when the value is found in sequence

not in - returns true when the value is not found in sequence.

operator.	Description
()	Parentheses (grouping)
**	Exponentiation
**x	Bitwise not
+x, -x	Positive, negative (unary +, -)
*, /, //, %	Multiplication, division, floor division, remainder.
+, -	Addition, subtraction
&	Bitwise and
^	Bitwise xor
	Bitwise or
<, <=, >, >=, <>, !=, ==, is, is not	Comparisons (relational operators), identity operators.
not x	Boolean NOT
And	Boolean AND
or	Boolean OR.

Expression combination of both operator and operands

Eg :- $>>> 4+5-2*3$

$= 3$

Eg :- $10/5*2*3$

$= 0$

Input output functions in Python :

Input	output
1) It is similar to the scanf statement in c.	1) In the output function you are using the print() function.
2) It always you enter the data in string format	2) It is similar to the printf statement in c.
3) The text in double quotes will be considered as normal text and display in the output screen	3) text in double quotes will be considered as normal text and display in the output screen.
Syntax:- input()	
Eg:- a=input()	Eg:- print("Hello")
a = 5	hello
→ string	>>> a=5
a="abc" → string	>>> print('a'=('a=',a))
a=5 → string	a=5
a=2.5 → string	>>> a="KLU"
	>>> print("Welcome to" + a)
Eg:- a=int(input())	Welcome to KLU
# b=int(a)	
print(a)	
print(type(a))	

Program:- WAPP addition of 2 numbers to print the sum

Note:- take input from user or keywords

```

a=int(input())      (or) a=int(input('enter the a value'))
b=int(input())      b=int(input('enter the b value'))
print(a+b)          print("addition of two numbers is =",a+b)

```

by using dot format specific

Input	Output
1) It is similar to the scanf statement in C.	1) In the output function you are using the print() function.
2) It always you enter the data in string format	2) It is similar to the printf statement in C.
3) The text in double quotes will be considered as normal text and display in the output screen	3) Text in double quotes will be considered as normal text and display in the output screen.
<u>syntax</u> :- input()	
Eg:- a=input()	Eg:- print("Hello")
a=5	Hello
→ string	>>> a=5
a="abc" → string	>>> print("a"=("a=", a))
a="c" → string	a=5
a=2.5 → string	>>> a="KLU"
	>>> print("Welcome to"
Eg:- a=int(input())	+a)
# b=int(a)	Welcome to KLU
print(a)	
print(type(a))	

Program:- WAPP addition of 2 numbers to print the sum

Note:- Take input from user or keywords

a=int(input())

b=int(input())

print(a+b)

(or) a=int(input("enter the a value"))

b=int(input("enter the b value"))

print("addition of two numbers

is = "a+b")

By using dot format specific

```
a=int(input("Enter a value:"))
b=int(input("Enter b value:"))
print("addition of {} and {} is: {}".format(a,b,a+b))
```

(or)

by using module format

```
print("addition of %d and %d is '%d'%."%(a,b,a+b))
```

Map(): - More than 1 input to take from the user by using the Map(). And also 5, 6, 7 and a, b, c

→ In the Map() it is used in list, tuple, dictionary.

split(): - It is used to split the value after you entered the data.

```
↳ a,b,c=map(int,input("Enter a,b,c values:"))
# print(a+b+c)
```

```
print("addition of %d and %d and %d is '%d'%."%(a,b,c,a+b+c))
```

Command line Arguments:-

The arguments that are given after the name of the program in the command line shell of the operating system are known as command line Arguments. If you run the program first you need to import the sys module. The sys module provides functions and variables used to manipulate different parts of the python runtime environment.

eg:- import sys

```
a=int(sys.argv(1))
b=int(sys.argv(2))
print(a+b)
print(len(sys.argv))
```

conditional statement: It is also known as a decision statements first you need to check the condition then statements is executed when the condition is true otherwise failed statement is executed. In the conditional statements it can following some set of statements.

- 1) If-statement
- 2) If-else statement
- 3) If-elif-else statement
- 4) Nested if statement (child leaf)

1) If-statement:-

If the condition is true the set of statements (are false) will be executed, if the condition is false no statement will be executed.

Syntax:-

```
if(condition):
```

```
#statements
```

```
#outer statements
```

```
int a = int(input())
```

```
b = int(input())
```

```
if a > b:
```

```
    print("Welcome to KLO")
```

```
    print("EEE")
```

Ex:-
a = 7
b = 10

```
if a < b:
```

```
    print("Welcome to KLO")
```

```
    print("EEE")
```

```
a = 7
```

```
b = 10
```

```
if a > b:
```

```
    print("Welcome to KLO")
```

```
    print("EEE")
```

Output :- Welcome to KLO

EEE

Output :- W EEE

2) If-else statement:-

If a condition is true the set of statements (if) are executed, if condition is fail else statement will be executed.

* Write a python program to check whether the person eligible or not for voting.

```
age = 18
```

```
if (age > 18)
```

```
    print("eligible for voting contest")
```

else :

```
print('not eligible')  
print("Welcome to Newworld")
```

```
age = int(input("Enter your age :"))
```

```
if (age > 18):
```

```
    print("eligible for voting contest")
```

```
else :
```

```
    print("NOT eligible")
```

```
    print("Welcome to Newworld")
```

*Write a python program to check the person is senior citizenship or not.

sample

```
age = 70
```

```
if (age > 60):
```

```
    print('person is belongs senior citizenship')
```

```
else:
```

```
    print("person is not belongs to senior citizenship")
```

```
    print("Hello")
```

run

```
age = int(input("Enter person age"))
```

```
if (age > 60):
```

```
    print('person is belongs to senior citizenship')
```

```
else:
```

```
    print("person is not belongs to senior citizenship")
```

```
    print("Hello")
```

3) If-elif-else statement :- If if condition is true the set of statements are executed ^{remaining elif & else are skipped} or elif block is checking if elif block also condition is true to checking the if condition, if if condition is failed then elif block is executed to skip the if, else block is executed.
if "else-if" both conditions are failed then else part is executed.

Syntax:-

if 'if' condition is true.

a = 5

if a > 0:

statements executed

elif a < 0:

statements executed

else:

statements executed

code after it executed

elif condition is true:-

a = -5

if a > 0:

statements executed

elif a < 0:

statements executed

else:-

statements executed

code after the if executed.

output: elif is true.

if "else" condition is true.

a = 0

if a < 0:

print("Hi")

elif

elif a > 0:

print("Hello")

else:

print("Bye")

Output: Bye

* Write a python program to find the largest of a given statement.

a = 11

b = 12

c = 5

if (a > b and a > c):

print("a is big")

elif (b > a and b > c):

print("b is big")

else (c > a and c > b):

print("Welcome")

print("c is big")

print("Welcome")

```

a = int(input())
b = int(input())
c = int(input())
if a > b and a > c:
    print("a is Big")
elif b > a and b > c:
    print("b is Big")
else c > a and c > b:
    print("c is Big")
    print("Welcome to Python")

```

Nested if :- If inside another if that is called nested if.

If the condition is true nested if conditions are executed if condition's false it come out of the nested if and else statements are executed. If the inner if condition is true the set of inner if statements are executed if the inner if statement is false inner else statements are executed. If the outer if false the inner if statements outer else part is executed.

Syntax:-

Outer if condition true	Outer if condition false
a = 5	a = -5
if a >= 0:	if a >= 0:
if a == 0:	if a == 0:
# statements executed	# statements executed
else :-	else :-
# statements executed	# statements executed
else:	else:
# statements executed	# statements executed
# code after if executed	# code after if executed

```

a=5
if a>0:
    if a==0:
print("welcome")
else if:
    print("eee")
else:
    print("cse")

```

10x)

```

a=0
if a>0:
    if a==0:
print("welcome")
else if:
    print("eee")
else:
    print("cse")

```

* Write a python program to check whether the year leap or not.

```

a = int(input("Enter a leap year"))
if (a%400==0) and (a%100==0):
    print(a, "is a leap year")
elif (a%4==0) and (a%100!=0):
    print(a, "is a leap year")
else:
    print(a, "is not a leap year")

```

Loops :-

It is nothing but a iterating the values until certain condition is met/reached.

While loop: The rules of while loop:

- > To evaluate the condition.
- > If the condition is true the while body statement is executed.
- > This process is continued until the condition is false.
- > Once the condition is false the loop is terminated.

Syntax :- While (condition)
 # body of while
 increment/decrement.

problem 1:- Write a program kLU 20 times by using while loop.

```
n=1
while(n<=20)
    print("kLU")
    n+=1
```

problem-2:- Write a program to print the sum of n natural numbers by using while loop.

```
a=int(input()) #10
if(a<0):
    print("enter positive numbers only")
else:
    sum=0
    while(a>0):
        sum=sum+a
        print("sum is: ",sum)
```

problem 3:- Write a program to print the multiplication of a table

```
a=int(input()) #10
```

```
c=1
while(c<=10):
```

```
b=a*c
```

```
print(a,"*",c,"=","b)
```

Problem 4:- Write a program to calculate the factorial of a given number.

```
a=int(input()) #10
```

```
i=1
```

```
while(a>0):
```

```
i=i*a
```

```
a-=1
```

```
print(i)
```

Problem 5:- Write a program to check whether the given number is palindrome or not.

```
a = int(input()) # 121
```

```
temp = a
```

```
r = 0
```

```
while (a != 0):
```

```
    r = r * 10 + a
```

```
    a /= 10
```

```
if (temp == r):
```

```
    print(temp, "is a palindrome")
```

```
else:
```

```
    print(temp, "is not a palindrome")
```

Nested while loop:-

→ While inside another while is called nested while loops

→ If the outer while loop condition is true to executed and will check the inner while loop condition.

→ If the inner while loop condition failed then it immediately goes to the outer while loop.

→ If outer while loop is failed it will terminate & come out of the loop.

while (condition):

statement

while (condition):

body of inner while loop

Problem 6:- Write a program to print pattern using while loop.

```
*
* *
* * *
* * * *
* * * * *
```

Sol:-

```
n = int(input())
```

```
i = 1
```

```
while (i <= n):
```

j=1

while (j<=i):

print("*",end=" ")

j+=1

print()

i+=1

for loop:-

It is like while loop, to initialize the value, to check the condition to update the value at same location. It is also working as a sequential manner i.e., iterating the value.

```
for i in range(n):
```

```
    // statement 1
```

```
    // statement 2
```

In this for loop you are using the sequence, instead of sequence we are using list, tuple, dictionary, range, methods. At this situation we are using the range method.

→ The Range Method is used to find where the value started and stopped, how much the difference b/w each and every digit (step size)

1) Write a program to print the KLU 20 times by using for loop.

```
a=input("enter a string") KLU
```

```
for i in range(20): 0-19
```

```
    print(a) KLU
```

2) Write a program to print the sum of n natural numbers.

```
n=int(input("enter a number"))
```

```
if (n<0):
```

```
    print("enter positive number only")
```

```
else:
```

```
    sum=0
```

```
    for i in range(1, n+1):
```

```
        sum+=i
```

```
print(sum)
```

3) Write a program to print the multiplication of a table.

```
n=int(input("Enter a number"))
```

```
i=1
```

```
for i in range(1,10):
```

```
print(n,"x",i,"=",n*i)
```

4) Write a program to print the Fibonacci series.

```
n=int(input("Enter a number"))#10
```

```
f=0
```

```
s=1
```

```
print(f,s)
```

```
for i in range(2,n+1):
```

```
temp=f+s
```

```
print(temp)
```

```
f=s
```

```
s=temp
```

```
f=2
```

```
s=2
```

```
temp=f+s
```

```
=4
```

```
f=2
```

```
s=3
```

```
temp=f+s
```

```
=5
```

Nested for loop:- for loop inside another loop that is called nested for loop.

Program:-

Write a program to print the pattern by using for loop.

```
*
*  *
*  *  *
*  *  *  *
*  *  *  *  *
```

```

n = int(input(" "))
for i in range(1, n+1):
    for j in range(1, i+1):
        print("*", end=" ")
    print()

```

Program:- Write a program to print the pattern by using for loop.

```

      *
     * *
    * * *
   * * * *
  * * * * *

```

```

n = int(input(" "))
for i in range(1, n+1): #outer for loop
    for j in range(n-i): #print row only
        print(" ", end=" ")
    for k in range(2*i-1):
        print("*", end=" ")
    print()

```

Program :- Write a program to print the pattern by using for loop.

P

P y

P y +

P y + h

P y + h 0

P y + h 0 n

n = input("enter a string")

a = int(input("enter the length of"))

for i in range(a):

for j in range(i+1):

print(n[i:j], end=" ")

print()

Jumping statements: The another name of jumping statements are loop control statements. We are working on the 3 types of loop control statements.

1. Break

2. Continue

3. Pass

1. Break: Break is used in looping statements like for loop and while loop. When you are performing the break statement in your program, certain condition is true then after coming out of the loop.

Syntax :- break

eg:- for i in range(10):

print(i)

if i == 5:

break

print("Bye")

O/P:

0

1

2

3

4

Bye

2) continue:- A also working like for loop and while loop when you are using the continue, only certain condition is skip to continue the remaining process until you reach certain limit.

Syntax :- continue.

Eg :- for i in range(10):

if (i==5): $\rightarrow$ skipped

continue

print(i)

print("Bye")

3) Pass:- When you are using the pass in the program there is no operation cannot be performed.

Syntax :- Pass

Eg :- for i in range(10):

if (i==5):

pass

print(i)

print("Bye")

String :-

Strings it is a collection of characters, then it is called as a string. It denotes single quotes or double quotes or triple quotes. In the python there is no character type then this character is also denoted as a string.

$\rightarrow$ If you want to take 1 character in a given string you are using indexing only.

type() :- When you are using the type method to print the original status of the variable.

eg:- >>> a = 'KLU'

>>> a

>>> a = "KLU"

>>> a

>>> a = '''KLU'''

>>> a

'KLU'

>>> print(type(a))

<class 'str'>

>>> a = 'e'

>>> print(type(a))

<class 'str'>

concatenating more than 2 strings:- the concatenating of 2 strings we are using the '+' operator. It means the 2nd string is attached to the 1st string.

eg 1:- a = input("Enter a string")

print("Hello" + a)

2. a = input("Enter a string")

b = input("Enter a string")

print(b + a)

Comparison of strings:-

You are performing the comparison of 2 strings by using "=="

eg:- a = input("Enter a string")

b = input("Enter a string")

if a == b:

print("Yes")

else:

```
print("NO")
```

length(): To find out the length of the string by using the length method.

```
a=input("enter a string")
```

```
b=input("enter a string")
```

```
print(len(a))
```

Indexing :- It you want to print the character of it in a given string you are using indexing only.

Eg:- #a=input("enter a string")

```
b=input("enter a string")
```

```
print(b[1])
```

The strings are immutable i.e, cannot be change any character of a given string. But if you change the entire string.

Python string slicing :- It means it will take and print the particular range of given string, where is the start position and the difference b/w start and stop positions (stepsize)

Syntax :-

```
Str = (start: stop: stepsize)
```

```
str = 0 1 2 3 4 5python  
-6 -5 -4 -3 -2 -1
```

```
print(str[:]) #python
```

```
print(str[::-1]) #nohtyp
```

```
print(str[1:3]) #yt
```

```
print(str[-1:-6]) #nohtyp
```

```
print(str[1:5:2]) #yh
```

* Write a python program to check whether the string is palindrome or not?

sol:- `str=input( )`

`if(str==str[::-1]):`

`print(str,"palindrome")`

`else:`

`print(str,"Not a palindrome")`

* Write a program to count the no. of spaces present using count method.

sol:- `str=input( )`

`b=" "`

`c=str.count(b)`

`print(c)`

* Write a python program to check whether the given string is palindrome or not without slicing method.

sol:- `str=input( )`

`r=""`

`for i in str`

`r=i+r`

`if(str==r)`

`print(str,"palindrome")`

`else:`

`print(str,"Not a palindrome")`

* Write a program to (print) check whether the character vowel or not.

`str=input( )`

`for i in ('a','e','i','o','u','A','E','I','O','U')`

`if(str==i)`

`print("vowel")`

else:

print("Not vowel")

(or)

str=input()

if str in ('a','e','i','o','u','A','E','I','O','U'):

print("vowel")

else:

print("Not a vowel")

* Vowel count consonant count.

str=input()

c=0 v=0

="aeiouAEIOU"

if str in a:

v+=1

else

c+=1

print(v)

print(c)

* Write a python program to print the capitalite each word first character in the given string.

str=input()

c=str.title()

print(c).

Functions:- CO-2

- 1) Every function should starts with def keyword
- 2) Every function should have name (not equal to any key word)
- 3) If you are passing any parameters or arguments is an optional, in function definition
- 4) Every function name should ends with colon operator.

function calling:-

- 1) Without function calling the output is not generated.
- 2) Just it simply define the function name.

Ex:-

```
def EEE(a,b): #definition
```

```
    print(a+b)
    print(a-b)
    print(a*b)
    print(a/b)
    print(a%b)
```

} user defined function

→ compile time

```
(# Main) EEE(3,4) # calling the function
```

(Or)

```
def EEE(a,b):
```

```
    print(a+b)
```

```
    print(a-b)
```

```
    print(a*b)
```

```
    print(a/b)
```

```
    print(a%b)
```

→ run time

```
X = int(input("enter a first number:"))
```

```
Y = int(input("enter a second number:"))
```

```
EEE(X,Y)
```

```
def EEE():
```

```
    a=10
```

```
    b=20
```

```
    a = int(input("enter a first number:"))
```

```
    b = int(input("enter a second number:"))
```

```
    print(a+b)
```

```
    print(a-b)
```

```
    print(a*b)
```

```
    print(a/b)
```

```
    print(a%b)
```

EE|)

* Write a python program to print the square of number by using the function with return.

```
def sq(x):
```

```
    return x*x
```

```
    print(sq(7))
```

```
    ̂
```

```
def sq(x)
```

```
    return x*x
```

```
x=int(input("Enter a number:"))
```

```
y=sq(x)
```

```
print(y)
```

* Write a python program to print the factorial of a given number using functions.

```
def factorial(x):
```

```
    if(x<0):
```

```
        return "Enter positive numbers only"
```

```
    elif (x==0 or x==1):
```

```
        return 1
```

```
    else:
```

```
        fact=1
```

```
        while(x>0): (or) for i in range(1,x+1):
```

```
            fact*=x
```

```
            x-=1
```

```
        return fact
```

```
        (or)
```

```
def factorial(x):
```

```
    if(x<0):
```

```
        return "Enter positive numbers only"
```

```
elif x==0 or x==1 :
```

```
    return 1
```

```
else :
```

```
    fact=1
```

```
    for i in range(1,x+1):
```

```
        fact*=x
```

```
    x-=1
```

```
    return fact
```

```
x = int(input("Enter a number:"))
```

```
print(fact(x))
```

* Write a python program to print the multiplication of a table.

```
def mul(x,mul=10):
```

```
    for i in range(1,mul+1):
```

```
        y = xi * i
```

```
        print("{} * {} = {}".format(x,i,y))
```

```
    return y
```

```
y = int(input())
```

```
a = multiplication(y)
```

```
for i in a:
```

```
    print(i)
```

* Write a python program to print the sum of a natural numbers using functions with return type.

```
def sum(n): # calling
```

```
    return int(n*(n+1)/2)
```

```
num = int(input())
```

```
print(sum(num))
```

* Write a python program to check whether the given number is palindrome or not by using functions with return type.

```
def pal(n):
```

```
    r=0
```

```
    while(n!=0)
```

```
        re = n%10
```

```
        r = r*10 + re
```

```
        n /= 10
```

```
    return r
```

```
1*10+2
```

```
12
```

```
num = int(input())
```

```
r = pal(num)
```

```
if (r == num):
```

```
    print(num, "is a Palindrome")
```

```
else:
```

```
    print(num, "is not a Palindrome")
```

Types of Arguments:-

- 1) Required Arguments
- * 2) Keyword Arguments
- 3) Default Arguments
- 4) Variable length Arguments

1) Required Argument :- The no. of Arguments should be same in both function call and function definition
→ They should be following the same order or same position

Ex:-

```
def klu(a,b):  
    print(a,b)  
klu(20,30)
```

2) Keyword Argument:- It is not following any order of position but initialization will be done based on the names.

Ex:-
def KLU(a,b):
 print(a,b) # 20,10
KLU(b=10,a=20)

3) Default Argument :- the no. of arguments need not to be match with both function call and definition

→ some of the Arguments will be considered as a default argument.

Ex:-

```
def KLU(name, course="EEE"):
```

Actual parameters
can't be changed

```
    print(name) # KLEF  
    print(course) # Mtech
```

```
KLU(name="KLEF", "M-Tech") # actual para
```

```
KLU(name="KLEF") # formal para .
```

```
    KLEF
```

```
    EEE .
```

4) Variable length Argument :- variable is a argument which is based on the no. of arguments.

If you are passing the '*' in function definition then it goes to the prefix of your variable length argument. By the placing of star(*) prefix to the argument of function definition

Ex:-
def KLU(*courses):

```
    for i in courses:
```

```
        print(i)
```

```
KLU("M-Tech", "B-Tech", "MBA")
```

Recursion

The function calling itself is called recursion. Means you are calling at same function. Based on two conditions recursion is working.

1) Base case :-

2) It terminates from recursion

2) Recursive condition:-

calling itself.

* Write a python program to print the factorial of a given number using recursion.

```
def fact(n):
```

```
    if n==0 or n==1:
```

```
        return 1
```

```
    else:
```

```
        return n*fact(n-1)
```

```
    num=int(input())
```

```
    print(fact(num))
```

* Write a python program to print the pattern by using functions

```

*
* *
* * *
* * * *
* * * * *
```

```
def pattern(n):
```

```
    if n==0:
```

```
        return
```

```
    else:
```

```
        pattern(n-1)
```

```
        print("*"*n)
```

```
    num=int(input())
```

```
    pattern(num)
```

List

- List are used to store the multiple items in a single variable
- List items are ordered it can be changeable and it can be allowing the duplicate values.
- By using the indexing we can print the one by one element in the given list.
- The List is defined by square brackets `[ ]`.

```
L1 = [1, 2, 3, 4, 5, 3+4j, 'python']
```

```
print(L1)
```

Runtime Initialization:-

- By using input method to read the elements at runtime.
- To create the empty list and to store the items or elements in the list by using append method.

Append Method:-

- Only single value or element is inserted at the list.

Syntax:-

```
listname.append(element)
```

```
scq = [ ]
```

```
for i in range(10)
```

```
    n = int(input())
```

```
    scq.append(n)
```

```
print(scq)
```

Accessing the element in the List:-

To print the particular element in the list by using indexing

Ex:-

```
L1 = [11, 22, 33, 44, 55, 3+4j, 'python']
```

```
print(L1)
```

```
print(L1[3])
```

Updating List :-

L1

without updating list :-

L1[3]

Delete() :-

By using delete() we can delete a particular element in a given list.

L1 = (1, 2, 3, 4, 5, 3+4j, 'python')

List slicing :-

print(L1[:1])

print(L1[-1])

print(L1[1:3])

print(L1[1:2])

Count() Method :-

L1 = (1, 2, 3, 4, 5, 3, 2, 1)

print(L1.count(1)) # 2

Length() Method :-

To print the length of the entire list by using len() method.

Syntax :-

len(listname)

print(len(l1)) # 8.

Extend Method:-

TO updating the my list more than one value by using extend method.

```
print(L.extend([1,2,3,4]))
```

Remove Method:-

Our list is an ordered list duplicate value is allowed so that (it is using to remove) to remove the element in the given list by using remove method by it removes the first occurrence of the given value from the list

syntax :- remove(value)

```
L1 = [1,2,3,4,4,3,2,1]
```

```
print(L1.remove(1))
```

```
print(L1) # [2,3,4,4,2,2,1]
```

```
print(L1.remove(1))
```

```
print(L1) # [2,2,4,4,3,2]
```

Reverse Method:-

TO print the reverse of a given list by using reverse method.

```
reverse()
```

```
L1 = [1,2,3,4]
```

```
print(L1.reverse())
```

```
[4,3,2,1]
```

Sort :-

The list is unordered list so that to print the values from ascending order by using sort method.

```
L1 = [5,4,3,1,2]
```

```
print(L1.sort())
```

```
[1,2,3,4,5]
```

POP() Method:-

To remove the last element of a given list by using the Pop Method.

```
L1 = [5, 4, 3, 1, 1, 2]
```

```
print(L1.pop())
```

```
print(L1) # [5, 4, 3, 1]
```

* Write a python program to print the sum of elements from a given list by using functions.

```
def KLU(List):
```

```
    return sum(List) # 15.
```

```
L1 = [1, 2, 3, 4, 5]
```

```
print(KLU(L1))
```

* WAPP to print the sum of elements using functions runtime initialization.

```
def KLU1():
```

```
    n = int(input("How many elements:"))
```

```
    L1 = []
```

```
    for i in range(n):
```

```
        ele = int(input())
```

```
        L1.append(ele)
```

```
    return sum(L1)
```

```
    print(KLU)
```

* WAPP to print the given number in a given list.

```
L1 = [1, 2, 4, 6, 5, 7, 9, 11, 4]
```

```
for i in L1:
```

```
    if (i % 2 == 0):
```

```
        print(i)
```

*WAPP to merge the two lists by using functions.

```
def klu(list1, list2):
```

```
    list1.extend(list2)
```

```
    return list1
```

```
a = [1, 2, 3, 4]
```

```
b = [5, 6, 7, 8]
```

```
print(klu(a, b))
```

*WAPP to swap the first and last elements of the list.

```
def klu(list):
```

```
    size = len(list)
```

```
    temp = list[0]
```

```
    list[0] = list[size-1]
```

```
    list[size-1] = temp
```

```
    return list
```

```
l1 = [5, 2, 3, 4, 1]
```

```
print(klu(l1))
```

*WAPP to combine two lists by using sorted method.

```
def klu(list1, list2)
```

```
    return sorted(list1+list2)
```

```
a = [1, 2, 5, 7]
```

```
b = [2, 4, 6, 8]
```

```
print(klu(a, b))
```

List comprehension :-
List comprehension is used to reduce the no. of programming lines.

$a = [\text{expression for variable in sequence}]$

↘

List []

Tuple ()

Set { }.

dict { }.

range()

*WAPP +.

$L = [1, 2, 3, 4, 5]$

$[1, 4, 9, 16, 25]$

~~$res = [i**2 \text{ for } i \text{ in } L]$~~

$res = [i**2 \text{ for } i \text{ in } L]$
 $\text{print}(res)$

Looping in List :-

```
L1 = [0, 1, 2, 3, 4, 5]
```

```
res = [i for i in L1]
```

```
print(res)
```

Combination of List Comprehension and looping

```
L1 = [1, 2, 3, 4]
```

```
L2 = [2, 3, 4, 5]
```

```
res = [(1, 2), (1, 3), (1, 4), (1, 5),
```

```
      (2, 3), (2, 4), (2, 5),
```

```
      (3, 2), (3, 4), (3, 5),
```

```
      (4, 2), (4, 3), (4, 5)]
```

```
res = [(x, y) for x in L1 for y in L2  
        if (x != y)]
```

```
print(res)
```

* WAPP to print the multiplication of a table by using List Comprehension.

```
n = 10
```

```
res = [n * i for i in range(1, 11)]
```

```
print(res)
```

```
10, 20, 30, ---
```

```
res = (print('1 2 3 4 5 6 7 8 9 10'))
```

Tuple:-

→ Tuple is immutable means we cannot change the elements and update in tuple and cannot be removed element

→ tuple denoted by parenthesis `{ }`

Create a tuple:-

Syntax:- Variable name = (elements) →

`a = (1, 2, 3, 4)`

`a = ( )` → tuple without element

`a = (1)` is not a tuple, it is an integer.

`a = (1,)` → It is a tuple.

Update a Tuple:-

update 1)

`t1 = (1, 2, 3, 4)`

`t2 = (3, 4, 5, 6)`

`t3 = t1 + t2`

`t4 = t2 + t1`

`print(t3)`

`print(t4)`

Removing tuple:-

`a = (1, 2, 3, 4, 5)`

`del a[0]`

`del a` - Entire tuple is deleted.

Accessing elements in tuple:-

`a = (10, 20, 30, 40, 50, 60, 70)`

String

`a[0:4]` o/p: 10, 20, 30, 40

Some basic operations in a tuple:-

$t_1 = (1, 2, 3, 4, 5)$

$t_2 = (2, 3, 4, 5, 6)$

$\text{len}(t_1) \# 5$

$\text{len}(t_2) \# 5$

Concatenation:-

$t_1 = (1, 2, 3, 4, 5)$

$t_2 = (5, 6, 7, 8)$

$\text{len } t = t_1 + t_2$

$\text{print}(t)$

Repetition:-

By using * symbol

$t_1 = (1, 2, 3, 4, 5)$

$t_2 = (2, 2, 4, 5, 6)$

$t_1 * 2$

Membership operator:-

$t_1 = (1, 2, 3, 4, 5)$

$t_2 = (2, 3, 4, 5, 6)$

$2 \text{ in } t_1 \# \text{true}$

$6 \text{ in } t_2 \# \text{false}$

$2 \text{ not in } t_2 \# \text{false}$

$8 \text{ not in } t_1 \# \text{true}$

Iterative statements:-

$t_1 = (1, 2, 3, 4, 5)$

$t_2 = (2, 4, 5, 6, 7)$

for ele in t_1 :

print(ele)

max(t_1) # 5

min(t_2) # 2

* To find the index

$t_1 = (1, 2, 3, 4, 5)$

$t_2 = (2, 3, 4, 5, 6)$

index(5) # 4

Count:-

$t_1 = (1, 2, 3, 4, 5)$

$t_2 = (2, 3, 4, 5, 6)$

t_1 .count(5) # 2

t_2 .count(6) # 1

Converting into tuple:-

By using the tuple method, we can convert the list into a tuple.

$L_1 = [1, 2, 3, 4, 5]$

print(L_1)

print(type(L_1))

$a = \text{tuple}(L_1)$

print(a) # (1, 2, 3, 4, 5)

print(type(a)) # tuple

* WAPP to print the sum of tuple by using list comprehension.

```
def KLU(a):
```

```
    res = [x for x in a]
```

```
    return sum(res)
```

```
n = (1, 2, 3, 4, 5, 6, 7)
```

```
print(KLU(n))
```

* WAPP to print the sort lists in tuple by using tuple method or sorted method.

```
a = ([2, 1, 3], [5, 4, 3], [1, 0, 2])
```

```
res = ([1, 2, 3], [3, 4, 5], [0, 1, 2])
```

```
res = tuple(sorted(x) for x in a)
```

```
print(res)
```

Set

Set is a mutable means we can add or remove elements from a set after it has been created. Once initializing the elements in the set those elements are immutable.

Ex:- $a = \{ \}$ -> Empty set

$a = \{1, 3\}$

$a = \{1, 'a'\} \# 1, a$

$a = \{1, 'a', 3\} \# 1, a$

$a = \{1, 'a', 1\} \# 1, a$

$a.add(2) \# 1, a, 2$

Updating the entire sets:-

$a = \{1, 2, 3\}$

$b = \{4, 5, 6\}$

$a.update(b) \# \{1, 2, 3, 4, 5, 6\}$

$b.update(a) \# \{4, 5, 6, 1, 2, 3\}$

$a = \{1, 2, 3\}$

$b = \{3, 4, 5\}$

$a.update(b) \# \{1, 2, 3, 4, 5\}$

To remove the elements in your set by using remove method or discard method. When you performing remove method if the element is not available in the set (KeyError). When you using discard method when element not available in set then simply it will go out of set.

$a = \{1, 2, 3, 4, 5\}$

$b = \{5, 7, 8, 9\}$

$len(a) \# 5$

$a.clear() \# \{ \}$

Set is a mutable means we can add or remove
a set after it has been created. Once initializing the elements
in the set those elements are immutable.

Ex:- $a = \{\}$ $\rightarrow$ Empty set

$a = \{1, 3\}$

$a = \{1, 'a'\} \# 1, a$

$a = \{1, 'a', 3\} \# 1, a$

$a = \{1, 'a', 1\} \# 1, a$

$a.add(2) \# 1, a, 2$

Updating the entire sets:-

$a = \{1, 2, 3\}$

$b = \{4, 5, 6\}$

$a.update(b) \# \{1, 2, 3, 4, 5, 6\}$

$b.update(a) \# \{4, 5, 6, 1, 2, 3\}$

$a = \{1, 2, 3\}$

$b = \{3, 4, 5\}$

$a.update(b) \# \{1, 2, 3, 4, 5\}$

To remove the elements in your set by using remove
Method or discard Method. When you're performing remove
Method if the element is not available in the set
(KeyError). When you're using discard Method when element
not available in set then simply it will go out of set.

$a = \{1, 2, 3, 4, 5\}$

$b = \{6, 7, 8, 9\}$

$len(a) \# 5$

$a.clear() \# \{\}$

$a.pop() \Rightarrow \{2, 3, 4, 5\}$

$max(a) \# 5$

$min(a) \# 1$

sort():- $a = \{4, 3, 2, 1, 0\}$

$sorted(a) \# \{0, 1, 2, 3, 4\}$

The Python Set operations:-

In set provides different builtin methods to perform some mathematical set operations

1) is subset()

2) is superset()

3) union()

4) intersection()

5) difference()

6) symmetric-difference()

1) is ex:-

$a = \{1, 2, 3\}$

$b = \{1, 2, 3, 4, 5, 6\}$

$a.issubset(b) \# True$

$a < b \# True$

$a.issuperset(b) \# False$

$a > b \# False$

$a.union(b) \# \{1, 2, 3, 4, 5, 6\}$

$a|b$

$a.intersection(b) \# \{1, 2, 3\}$

$a \& b$

$a.difference(b) \# \{4, 5, 6\}$

$a-b$

$b.difference(a) \# \{4, 5, 6\}$

$b-a$

$a = \{1, 2, 3, 4, 5, 6\}$

$b = \{1, 2, 3, 7, 8, 9\}$

$a \cdot \text{symmetric-difference } b$ $(b) \# \{4, 5, 6, 7, 8, 9\}$
 $a \cap b$
 $b \cdot \text{symmetric-difference } a$ $(a) \# \{7, 8, 9, 4, 5, 6\}$
 $b \cap a$

* WAPP to find the even numbers of a given set and also apply the sort method.

$a = \{1, 2, 3, 4, 12, 14, 5, 8, 7, 6\}$

$(= \{2, 4, 12, 14, 8, 6\})$

$= \{2, 4, 6, 8, 12, 14\}$ - output.

$res = \{ele \text{ for } ele \text{ in } a \text{ if } ele \% 2 == 0\}$

$b = \text{sorted}(res)$

$\text{print}(b)$

(08)

Without list comprehension:-

create:-

dic1 = {} $\rightarrow$ Empty dictionary

syntax:-

dic1 = {key1: value1, key2: value2, key3: value3}

dic1 = {key1: value1,
key2: value2,
key3: value3}

By using dict method the tuple pair is converted into dictionary.

L1 = [("Name", "KLU"), ("Branch", "CSE"), ("RegNO", 79)]

dic1 = dict(L1)

{ "Name": "KLU", "Branch": "CSE", "RegNO": 79 }

Create a dictionary by using dictionary comprehension.

1) Expression

2) Sequence

3) Condition (optional)

Ex:- dic1 = {x: x**2 for x in range(5)}

{0:0, 1:1, 2:4, 3:9, 4:16}

key value.

(When you are)*

Accessing the elements:-

* When you are using accessing the elements with the help of key you can access the corresponding value.

dic1 = {"Name": "KLU", "RegNO": 79}

dic1["Name"] # KLU

dic1["RegNO"] # 79

09/08/24
Saturday

```
d1 = {"Name": "klu", "Branch": "EEE",  
      "subject": "python"}
```

```
Print(-type(d1))
```

```
d2 = d1.copy()
```

```
Print(d2)
```

```
Print(-type(d2))
```

o/p

class - 'dict'

```
{"Name": "klu", "Branch": "EEE", "subject":  
 "python"}
```

class - 'dict'

items() → key and value pairs, (tuple pairs)
→ to display the key value pairs of your dictionary it looks like as a tuple.

o/p Print(d1.items())
(('Name': 'klu'), ('Branch': 'EEE'),
 ('subject': 'python'))

keys() → To display the ~~in~~ keys in ^{your} dict

```
Print(d1.keys())
```

dict-key ('Name', 'Branch', 'subject')

values() → to display the values of in your dictionary.

```
print(d1.values())  
dict-values ('klu', 'EEE', 'python')
```

`pop()` $\Rightarrow$ based on the key item is deleted.

```
Print(d1.pop("Name"))
```

o/p $\Rightarrow$ klu.

```
print(d1)
```

o/p `{"Branch": "EEE", "subject": "python"}`

`print(d2)` $\rightarrow$ no change

`popitem()` $\rightarrow$ To Remove the last inserted item in the dictionary.

```
print(d2.popitem())
```

```
{'subject': 'python'}
```

`update()` $\rightarrow$ more than 1 we can insert to insert key value at the end of dict

```
d2.update({'Na': 'kl'})
```

```
print(d2)
```

write a python program to print the keys, values, items in your dict by using iterations.

Soln $\Rightarrow$

```
for i in d1:
```

```
    print(i) # name
```

```
    # Branch
```

```
    # subject
```

to find values using iterations

```
for j in d1.values:
```

```
    print(j) # klu
```

```
    # EEE
```

```
    # python.
```

for j in d.items():
 print(j) #



clear() ⇒

```
d1.clear()
print(d1)
# {}
```

dictionary won't be
 deleted in memory just it
 removes the keys and values

but del() will delete the
 dict in the memory

del() delete function

```
del d1
```

Print(d1) # error d1 is not defined that
 means d1 is deleted in the memory

update, changing or updating the value

```
d1["Name"] = "Key"
```

```
print(d1)
```

loops

funct

recursion

tuple

dict

set

findout the error

temp converts

ST CI pattern

simple or complex

inher

electrical bill generation
 app oriented qstn in list
 tuple, set, dict

nested loops
 list comprehensions

lists

- ① The lists are ordered.
- ② Elements of the list can be accessed by index.
- ③ List can store various types of elements.
- ④ Lists are mutable.
- ⑤ Lists allow duplicate elements.

~~For~~

Tuples

- ① tuples are ordered
- ② accessed by Index
- ③ can store various type of elements
- ~~④~~ ④ unmutable
- ⑤ allow duplicates

~~Index~~

+ index (elem)

+ [range] slicing

+ t₁ + t₂

+ * 2

+ count (ele)

Dictionaries

- ① ordered
- ~~②~~ ② can't accessed by index
- ③ store various types of elem
- ④ Mutable
- ~~⑤~~ ⑤ Don't allow duplicates

update (c5: "banana")

Class & Object:-

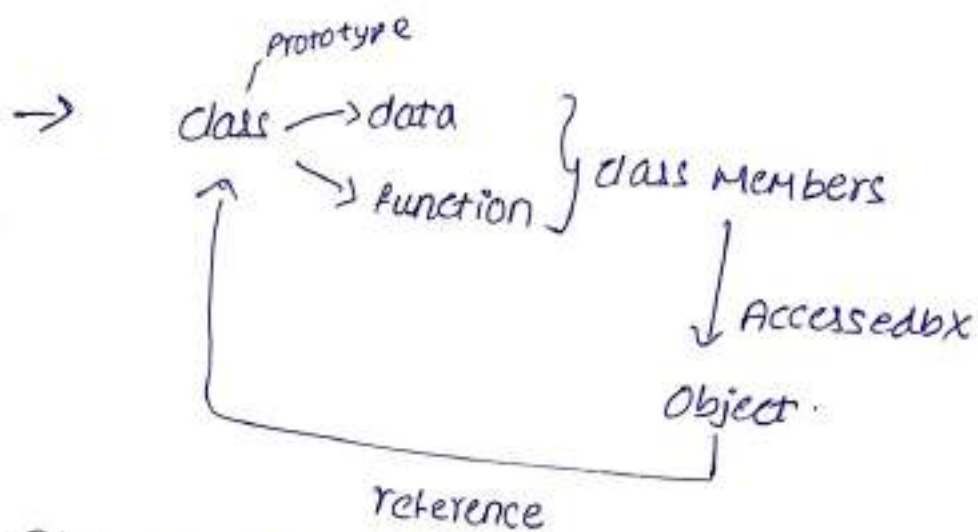
- The class is there is like an object constructor or a guide for creating an objects.
- A class is a user defined data type that contains both data and functions
- Then data and functions is called as class members.
- The class serves as a template to create objects.

Syntax:-

class class-name:

data

Functions



Ex:-

class student:

roll no = 23

name = "KLO"

branch = "EEE"

S1 = student1)

print("Roll no =", S1.roll no)

print("Name =", S1.name)

print("Branch =", S1.branch)

ex: Accessing by using objects at a time:-

```
class Student  
roll no = 23  
name = "KLW"  
branch = "EEE"
```

```
s1 = Student()
```

```
s2 = Student()
```

```
print("roll no =", s1.rollno)
```

```
print("Name =", s1.name)
```

```
print("Branch =", s1.branch)
```

```
print("Roll no =", s2.rollno)
```

```
print("Name =", s2.name)
```

```
print("Branch =", s2.branch)
```

Without object creation we can access the class members data by the class name.

class student:

```
roll no = 23
```

```
name = "KLW"
```

```
branch = "EEE"
```

```
print("Roll no =", student.rollno)
```

```
print("Name =", student.name)
```

```
print("Branch =", student.branch)
```

Constructor:

The constructor is ~~following~~ ^{type} ~~one~~ special method that method is -- init -- Method ~~is~~ is used to initialize the members of the class whenever we create an instance.

Syntax:-

```
def __init__(self, argument values)
```

```
self.member = argument value
```

Types of constructors:

- 1) Parameterless constructor
- 2) Parameterized constructors

Types of constructors:

- 1) Parameterless constructor
- 2) Parameterized constructor

1) Parameterless constructor:

```
class student:
```

```
    def __init__(self):
```

```
        print("Welcome to KLO")
```

```
s1 = student()
```

```
s2 = student()
```

2) Parameterized constructor:

```
class student:
```

```
    def __init__(self, name, roll):
```

```
        self.name = name
```

```
        self.roll = roll
```

```
    def display(self):
```

```
        print("Name =", self.name)
```

```
        print("Roll no =", self.roll)
```

```
s1 = student("KLO", 73)
```

```
s1.display()
```

```
s2 = student("KLEF", 76)
```

```
s2.display()
```

Self :- The self is a parameter is reference to the current instance of the class and is used to access variables that belongs to the class.

Class Methods:-

How to add methods in my class:

class faculty:

NAME = "RAM"

Empid = 892

def read(self):

print("Welcome to RW")

def write(self):

print("Writing---")

f1=faculty()

print("Name is:", f1.NAME)

print("Emp-Id is:", f1.Empid)

f1.read()

f1.write()

Subscribers

class student:

subscribers = 0

def __init__(self, name, id):

self.name = name

self.id = id

def display(self, topicname):

print("Name=", self.name, "Id=", self.id, "Today's
topic name is ", topicname)

def update_subscribers(self, sub):

self.subscribers += 1

self.subscribers += 1

s1=student("sai", 73)

s1.display("class & objects")

s1.update_subscribers("RAM")

print("My channel subscribers are:", s1.subscribers)

```

s2 = student("Sony", 39)
s2.display("Python")
s2.update_subscribers("ANAND")
print("My channel subscribers are: ", s2.subscribers)

```

Inheritance:

Acquiring property

- 1) Single inheritance
- 2) Multiple inheritance
- 3) Hierarch. inheritance
- 4) Multiple inheritance

Inheritance

It is the one of the feature of Object Oriented programming

* According to the features the derived class (child) can inherit or access the properties and methods of the base class (parent)



Syntax:-

class 'Base class':

Base class structure

class Derived class (base):

Derived class structure.

Single Inheritance Example Program.

class parent:

```
def pdisplay(self):  
    print("welcome to KLU!!")
```

```
class child(parent):  
    def cdisplay(self):  
        print("Hello")
```

```
C = child()
```

```
C.cdisplay()
```

```
C.pdisplay()
```

Ex:

```
class car:
```

```
def cdisplay(self):  
    print("A car has four wheels")
```

```
class BMW(car):
```

```
def Bdisplay(self):  
    print("E6514")
```

```
B = BMW()
```

```
B.cdisplay()
```

Ex:

```
class car:
```

```
def wheels(self):  
    print("The car has 4 wheels")
```

```
class Benz(car):
```

```
def Model(self):  
    print("The Benz Model is E-class")
```

~~B = Benz()~~

```
B.wheels()
```

```
B.Model()
```

Single inheritance using constructor

```
class teacher():
```

```
    def __init__(self, name, id):
```

```
        self.name = name
```

```
        self.id = id
```

```
    def tdisplay(self):
```

```
        print("Name=", self.name)
```

```
        print("Id=", self.id)
```

```
class lec(teacher):
```

```
    def __init__(self, name, id, age):
```

```
        self.name = name
```

```
        self.id = id
```

```
    def lecdisplay(self):
```

```
        print("Name=", self.name)
```

```
        print("Id=", self.id)
```

```
        print("Age=", self.age)
```

```
lec = lecturer()
```

```
lec = L("KLU", 78, 25)
```

```
lec.lecdisplay()
```

```
lec.tdisplay()
```

2) Multilevel inheritance:-

When you inheritance a derived class from another derived class.

* We have a class 'A', this class is our 'base class'.

* We have another class 'B', this class extend from base class A.

* We have another class 'C', this class extend from class B.

1. class grandfather():

```
def gdisplay(self):
```

```
    print("Grandfather class")
```

class parent(grandfather):

```
def pdisplay(self):
```

```
    print("Parent class")
```

class child(parent):

```
def cdisplay(self):
```

```
    print("child class")
```

```
c=child()
```

```
c.cdisplay()
```

```
c.pdisplay()
```

```
p=parent() g=grandparent()
```

```
p.pdisplay() g.gdisplay()
```

```
p.gdisplay() g.display()
```



2. -

class car():

```
def Model(self):
```

```
    print("BMW MS")
```

class showroom(car):

```
def showroom(self):
```

```
    print("Vizag")
```

class available(showroom):

```
def available(self):
```

```
    print("Only 1 car is available")
```

```
a=available()
```

```
a.available()
```

```
a.showroom()
```

```
s=showroom()
```

```
s.showroom()
```

```
s.Model()
```

Multilevel Inheritance using constructor:-

```
class car():
```

```
    def __init__(self, Model):
```

```
        self.Model = Model
```

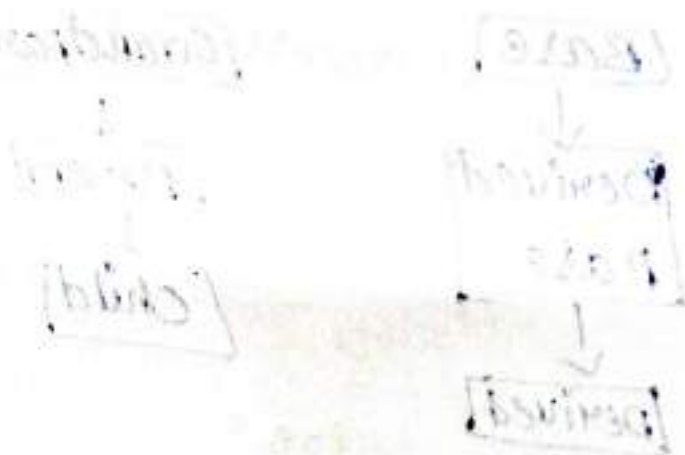
```
    def display(self):
```

```
        print("Model=", self.Model)
```

```
class showroom(car):
```

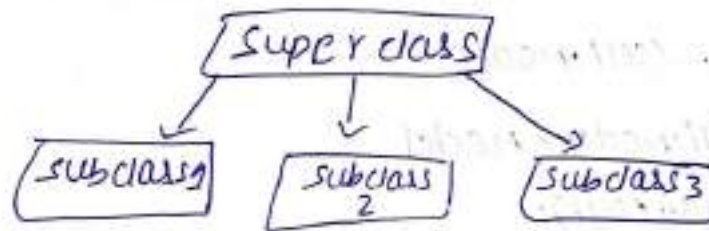
```
    def __init__(self,
```

→ showroom -> car
car -> car
car -> car



3) Hierarchical inheritance:

To acquiring the properties one parent class
More than 1 child.



Syntax:

```
class Superclass:
```

```
    #statements
```

```
class subclass1(Superclass):
```

```
    #statements
```

```
class subclass2(Superclass):
```

```
    #statements
```

```
class parent:
```

```
    def show(self):
```

```
        print("Parent class")
```

```
class child1(parent):
```

```
    def display(self):
```

```
        print("subclass1 or child1")
```

```
class child2(parent):
```

```
    def display(self):
```

```
        print("subclass2 or child2")
```

```
c = child1()
```

```
c.display()
```

```
c.show()
```

```
print("## ## Child class 2 Data ## @@@")
```

```
c1 = child2()
```

```
c1.display()
```

```
c1.show()
```

* Write a python program to take one car class, 3 child models by using hierarchical inheritance using constructor.

A) Class car:

```
def __init__(self, Model):  
    self.Model = Model  
def display(self):  
    print("Model =", self.Model)
```

class audi(car):

```
def __init__(self, Model):  
    self.Model = Model  
def display(self):  
    print("Model =", self.Model)
```

class benz(car):

```
def __init__(self, Model):  
    self.Model = Model  
def display(self):  
    print("Model =", self.Model)
```

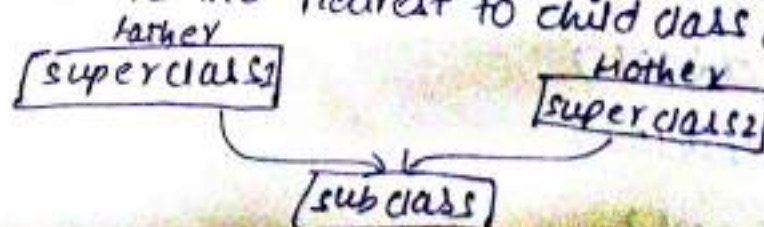
class bmw(car):

```
def __init__(self, Model):  
    self.Model = Model  
def display(self):  
    print("Model =", self.Model)
```

M = bmw("Red", "E6")

M.display

Multiple inheritance:- Both super class 1 and super class 2 inheritance to the nearest to child class (or) subclass.



both java and C++ doesn't support multiple inheritance but python does.

syntax:- class superclass1:

 #statements

class superclass2:-

 #statements

class subclass(superclass1, superclass2)

 #statements

=>

```
class father:
```

```
    def fdisplay(self):
```

```
        print("Father class")
```

```
class mother:
```

```
    def mdisplay(self):
```

```
        print("Mother class")
```

```
class child(father, mother):
```

```
    def cdisplay(self):
```

```
        print("child class")
```

```
c = child()
```

```
c.cdisplay()
```

```
c.mdisplay()
```

```
c.fdisplay()
```

Write a python program

④ Multiple Inheritance: * doesn't work in C or Java.

Both superclass1, superclass2 inheritings by the childclass or childclass.

Syntax:

class superclass1:

statements

class superclass2:

statements

class subclass (superclass1, superclass2):

statements

Father
superclass1

Mother
superclass2

Subclass

child / derived

Program:

class father:

def fdisplay(self):

print("Father class")

class Mother:

def mdisplay(self):

print("Mother class")

class child(father, mother):

def cdisplay(self):

print("child class")

c = child()

c.cdisplay()

c.mdisplay()

c.fdisplay()

supe

Access Modifiers

hile
class

by

-According to oop language like c, c++, java, python control the access modifiers by ones which are used to reset or restrict access to the variables and methods of the class.

⇒ It has three forms of access modifiers

hile

3

① Public Access Modifier

② Protected " " "

③ Private " " "

If you are not providing any access specifier then the program goes to the default access modifier i.e public.

→ Python uses single hyphen (-) to determine the access control for specific data member or member function of a class.

⇒ If you are using the double hyphen (--) then members of a class i.e are declare private accessible within the class only.

Program

output
welcome
10
10

```
class parent :
```

```
    a = 10 ;
```

```
    def display (self):
```

```
        Print ("welcome")
```

```
p = parent()
```

```
p.display()
```

```
Print (p.a)
```

```
Print (parent.a)
```

s. sdisplay()
 s. bdisplay()
 b = benz()
 b. bdisplay()
 b. cdisplay()

Write the same above program by using constructors.

class parent: Protected

-a = 10;
 def _display(self):
 print("welcome to klu")
 print(self._a)
 def show(self):
 parent._display(self)
 s = parent()
 s.show()

Private

class parent:

__a = 10;
 def __display(self):
 print("welcome")
 print(self.__a)
 def show(self):
 parent.__display(self)
 p = parent()
 p.show()

Write a python program to Restrict the data using access specifiers.

Ans →

```
class car:
    def __init__(self, model, colour, fueltype):
        self.model = model
        self._color = color
        self.__fueltype = fueltype

    def __display(self):
        Print("colour = ", self._color)
        Print("fuel-type = ", self.__fueltype)

    def show(self):
        Print("Model = ", self.model)
        car.__display(self)

c = car("MS", "Red", "Petrol")
# c._color = "white"
# c._model = "MS1"
c.show()
```

ENCAPSULATION

Wrapping data
Combining data

It describe the wrapping data and the methods that work on data within one unit.

It puts restrictions on accessing variables (attributes) and methods and can prevent

the accidental modification of data.

In the encapsulation used to hiding the data.

By using the sets and gets we are writing the program by using encapsulation

sets means $\rightarrow$ to initialising the data
gets means $\rightarrow$ to Return the data.

```
class parent;
```

```
--a = 10;
```

```
def setA(self, b):
```

```
    self.--a = self.--a + b
```

```
def getA(self):
```

```
    return self.--a
```

```
P = parent()
```

```
P.setA(30)
```

```
Print (P.getA)
```

26/03/24

POLY/MORPHISM

Many

Forms

$\Rightarrow$ The word polymorphism is it is a combination of two greek words means many-forms.

$\Rightarrow$ The polymorphism means same function name being used for different types.

$\Rightarrow$ The key difference is the data types, no. of alphabets used in a function.

eg: len()

```
Print (len("ktu")) #3
```

```
Print (len([10, 20, 30, 40])) #4
```

POLYMORPHISM:-

The word polymorphism means its a combination of two greek words means many forms.

→ The polymorphism means same function name being used for different types.

→ The key difference is the data types and no. of arguments used in a function.

Ex:-
`len()`
`print(len("KLO")) #3`
`print(len([10, 20, 30, 40])) #4.`

* Write a python program to take one parent class and father class to work on the polymorphism.

class parent: (class with functions)

def display(self):

print("Welcome to KLO")

def show(self):

print("Welcome to EEE branch")

class Father:

def display(self):

print("Welcome to KLEF")

def show(self):

print("Welcome to AIDS Branch")

P = parent()

F = Father()

for name in (P, F):

name.display()

name.show()

* Write a python program to take
class to work on polymorphism.

class cat:

def __init__(self, name, age):

self.name = name

self.age = age

def display(self):

print("Name =", self.name)

print("Age =", self.age)

def sound(self):

print("The cat sound is : Meow")

class dog:

def __init__(self, name, age):

self.name = name

self.age = age

def display(self):

print("Name =", self.name)

print("Age =", self.age)

def sound(self):

print("The dog sound is : Bow")

name = input("The cat name is")

age = int(input("The dog age is"))

d = dog(name, age)

for animal in (d, c):

animal.display()

animal.sound()

polymorphism

operator
overloading

method
overloading

method
overriding

1) operator overloading

The best example of this is to take unary operator '+' and its functionality is same but it shows different forms means sometimes it's working on the integers by taking integer type value and it also works on the concatenation of two strings.

Ex:-

```
a=20
```

```
b=30
```

```
print(a+b)
```

```
a="KLU"
```

```
b="Welcome"
```

```
print(a+b)
```

2) Method overloading:-

Two or more methods have the same name but different no of parameters or types or both methods. These methods are called method overloadings like other language java and c++ supports method overloading concept and python it doesn't support method overloading default.

* But there are different ways to achieve the method overloading in a python.

* The problem with the method overloading in python is that we may overload the methods but can only use latest defined function.

Ex:-

```
class Methodoverloading:
```

```
def display(self, a=None, b=None):
```

```
    print(a, b)
```

```
m = Methodoverloading()
```

```
m.display()
```

```
m.display(5)
```

```
m.display(5, 6)
```

def product(a,b):

p = a * b * 2

print(p)

def product(a,b):

p = a * b * 3

print(p)

p = product(2,3)

last statement is
working

O/p:- 18

note:-

Python does not support the Method overloading

Run-time

Method overriding:-

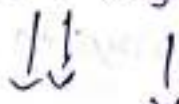
It is an ability of any object oriented programming language that allows a sub class or child class to provide a specific implementation of a method i.e. provided by the super class or parent class then the method sub class is said to ^{ride} over write the method in the super class.

Superclass

F → Transport → cycle

Subclass

S → Transport → car



Ex

class Father:

def transport(self):

print("Father using cycle")

class Son(Father):

def transport(self):

print("son using car")

s = son()

s.transport()

O/p:- son using car.

Class Father:

```
def transport(self):  
    print("Father using cycle")  
  
pass  
  
s = son()  
s.transport()
```

Difference b/w Methodoverloading and Methodoverriding

Methodoverloading	Methodoverriding
-------------------	------------------

- | | |
|--|---|
| 1) It is used to increase the readability of the program | 1) It provides a specific implementation of the method already in the parent class. |
| 2) It is performed within the same class | 2) It involves multiple classes |
| 3) Parameter must be different. | 3) The parameter must be same |
| 4) It is an example of compile time polymorphism. | 4) It is an example of runtime polymorphism. |

Super():-

In python super() is used to refer to the parent class or superclass

* It allows to call methods defined in the superclasses from sub classes.

* Its functionality inherited from superclass

* It returns the a proxy object which represents the super class.

Syntax:- super()

* There is no need of self in the super() class constructor, super class function (also) towards the sub class.

class car:

```
def __init__(self, Model):
```

```
    self.Model = Model
```

```
def display(self):
```

```
    print("Model = ", self.Model)
```

class audi(car):

```
    def __init__(self, Model, Model1):
```

```
        super().__init__(Model)
```

```
        self.Model1 = Model1
```

```
    def display(self):
```

```
        print("Audi Model is: ", self.Model1)
```

```
        super().display()
```

```
A = audi("suzuki", "RS")
```

```
A.display()
```

Nested class

Class inside another classes is called nested classes or inner classes.

* Without existing one type of object there is no chance of existing another type of object then we should go for the inner classes.

* Hiding code is another use of nested classes. You can hide the nested classes from the outside world.

* It is easy to understand classes

* Inside a class we can declare any no. of inner classes.

Ex:- Without parents --> No child

Without other object --> No existing inner object.

class parent:

```
    def display(self):
```

```
        print("Welcome to Parent class")
```

```
    class child:
```

```
        def display(self):
```

```
print("Welcome to child class")
```

```
parent().child().display()
```

Qp:- Welcome to child class.

```
class parent:
```

```
    def display(self):
```

```
        print("Welcome to parent class")
```

```
class child:
```

```
    def display(self):
```

```
        print("Welcome to child class")
```

```
p = parent()
```

```
p.display()
```

```
p1 = p.child()
```

```
p1.display()
```

Qp:- Welcome to parent class

Welcome to child class

```
class parent:
```

```
    def display(self):
```

```
        print("Welcome to parent class")
```

```
class child:
```

```
    def display(self):
```

```
        print("Welcome to child class")
```

```
class child1:
```

```
    def display(self):
```

```
        print("Welcome to child2 class")
```

```
p = parent()
```

```
p.display()
```

```
p1 = p.child()
```

```
p1.display()
```

```
p2 = parent.child.child2()
```

```
p2.display()
```

Q/p:- Welcome to parent class

(, child class

11 child class.

Data Abstraction:-

abstraction is used to hide the ir

Functionality of function from user.

* The users only interact with the basic implementation of the function but inner working is hidden

ex:-

The best example of data abstraction is: TV remote. When we use TV remote to increase the volume we don't know how pressing a key increases the volume of the TV, we only know how to press the + symbol of TV remote and automatically my TV volume is increased.

If you're accessing into the abstraction of the classes first you are importing the ABC module to use the abstraction in the python program.

Syntax:-

```
from abc import ABC
```

```
class class-name(ABC)
```

Abstract Base classes:-

It is an common application program of the interface for set of subclasses

- > It can be used by the third party application, which will provide the implementation such as plug-ins
- > It is also beneficial when we work with the large code base hard to remember all the classes.
- > An abstraction class can contain the both the normal and abstract methods.
- ** -> An abstract cannot be create a object for the abstract class.

ex:

```
from abc import ABC, abstractmethod
```

```
class parent(ABC):
```

```
    pass
```

```
class child1(parent):
```

```
    def display(self):
```

```
        print("Welcome to child1 class")
```

```
class child2(parent):
```

```
    def display(self):
```

```
        print("Welcome to child2 class")
```

```
c=child1()
```

```
c.display()
```

```
c1=child2()
```

```
c1.display()
```

With constructor

Static (compile time)
Method

```
from abc import ABC, abstractmethod
```

```
class parent(ABC):
```

```
    pass
```

```
class child1(parent):
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def display(self):
```

```
        print("child1.name =", self.name)
```

```
        print("child1.age =", self.age)
```

```
class child2(parent):
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
    def display(self):
```

```
        print("child2.name =", self.name)
```

```
        print("child2.age =", self.age)
```

C1 = child1("Rahul", 3) # static method

C1.display()

C2 = child2("Manoj", 4)

C2.display()

o/p: print both (child 1 & 2)
child 2 detail

Dynamic Method (run time)

for i in (child1, child2):

name = input("enter name")

age = int(input("enter age"))

k = i(name, age)

k.display()

o/p: last one

Passing a object as a parameter and returning the value also.

class A:

def GetNum(self):

self.x = int(input("Enter value of x:"))

self.y = int(input("Enter value of y:"))

def PutNum(self):

print("value of x = ", self.x)

print("value of y = ", self.y)

def Add(self, k):

S = A()

A.x = self.x + k.x

A.y = self.y + k.y

return S

Obj 1 = A()

Obj 2 = A()

print("Enter values of Object 1")

Obj1.GetNum()

Obj2.print("Enter values of Object 2")

Obj2.GetNum()

Obj3 = Obj1.Add (Obj2)

Obj1.Put Num ()

Obj2.Put Num ()

call by value and call by reference

call by value: (call by object).

When you are changing the formal parameter or update my formal parameter there is no reflection in actual parameter.

Ex:- def product(a):

print(a)

a += 10

print(a)

num = 10 -

print(num)

product(num)

print(num)

O/p:- 10
10
20
10

call by reference:-

When you are updating formal parameter the actual parameter is also updated, it means reflection is occurred.

Ex:- def func(list):

list.append(77)

print(list)

Call by Value Call by Reference

Call by Value:

Varshini@2300060016

When you are changing into the formal parameter or update there is no reflection in actual parameter i.e. called call by value or call by reference.

eg: def product(a):

print(a)

a+=10

print(a)

num = 10

print(num)

product(num)

print(num)

O/P

10

10

20

10

Call by Reference

When you are updating in formal parameter the actual parameter is also updated & the reflection is occurred.

def func(list):

list.append(79)

print(list)

O/P

list1 = [20, 30, 40, 60]

[20, 30, 40, 60]

func(list1)

print(list1)

[20, 30, 40, 60]

GOOD BYE TO CO-03

Files:-

The file is a named location used for storing data. If you are accessing into my file you can perform into the some actions in different files.

- 1) text file
- 2) Binary file

Opening a file:-

If you open a file first you have to perform any operation on it we use the open method.

Syntax:-

file object = open("filename", "Access mode")

└─ File pointer

→ The main user responsibility is to close my file also when you open it.

→ By using the close method if you are using file object or file pointer only.

Syntax:- file object.close()

Access modes:-

If you are performing into some access mode in my file also

) Read Method () → r

) Write Method () → w

) Append Method () → a

) Read Mode ():-

If the file is does not exist in my file location you can't performing the read mode.

The file is available only for reading the content in file.

If the file pointer points at the beginning of the file.

9/04/2024

CO-4

Files:-

The file is a named location used for storing data. If you are accessing into my file, you can perform into the same actions in different files.

- 1) Text file
- 2) Binary file

Opening a file:-

If you open a file, first you ^{can} have to perform any operation on it. We use the open method.

Syntax:-

file_object = open("filename", "Accessmode")

→ file pointer

→ The main user responsibility is to close my file also when you open it.

→ By using the close method, if you are using file object or file pointer only.

Syntax:- file_objt.close()

Access Modes:-

If you are performing into some access modes in my file also

- 1) Read Method () → r
- 2) Write Method () → w
- 3) Append Method () → a

1) Read Mode:-

If the file does not exist in my file location, you can't perform the read mode.

→ The file is available only for reading the content in file.

→ If the file pointer points at the beginning of the file.

→ The file should exist b/w before opening in read mode

Ex:- abc.txt

```
fp = open("abc.txt", "r")
```

```
fp.close()
```

abc.txt

Welcome to KLU

2) Write Mode):-

→ Only for writing the content.

→ The file pointer points at the beginning of the file.

→ If the file is exist then it is opened and pointer points at beginning and data will be overwritten.

→ If the file does not exist then new file will be created with the given file name.

Ex:- abc.txt

```
fp = ("abc.txt", "w")
```

```
fp.close()
```

abc.txt

Welcome to KLU

3) append mode ():-

* To append and write the data.

* If the file is exist the file pointer points at the end of the content file.

* If file does not exist new file will be created and the file pointer ^{points} at the beginning.

Ex:- abc.txt

```
fp = ("abc.txt", "a")
```

```
fp.close()
```

File exist

abc.txt

Welcome to KLU

abc.txt

does not exist

Access Mode .

- text files
- rb -> reading content
 - wb -> writing content
 - ab -> appending of the content
 - rb+ -> reading & writing -> file pointer points at the beginning
 - wb+ -> reading & writing -> file pointer points at the beginning
 - ab+ -> reading & appending -> file pointer points at the existing data.

Ex:-

- 1)

```
fp = open("abc.txt", "w")
fp.write("Hello world!!")
fp.close()
```
- 2)

```
fp = open("abc.txt", "r")
print(fp.read())
fp.close()
```
- 3)

```
fp = open("abc.txt", "a")
fp.write("Welcome to EEE")
fp.close()
```
- 4)

```
fp = open("abc.txt", "r") # Reading content by using bit positions
print(fp.read(5))
print(fp.read(4)) -> bit positions
fp.close()
```

Qp:- Welco
me-to

- `r+ mode` :-

If the file is available in file location you can read perform the reading & writing of your file.

```
fp = open("abc123.txt", "r+")
```

```
print(fp.read())
```

```
fp.close()
```

or `W+ mode`

```
fp = open("abc123.txt", "w+")
```

```
fp.write("Hello world")
```

```
fp.close()
```

or `W+ mode`

`W+ mode` :-

reading, writing and creating.

If (writing) the file is not available in the file location initially to create the file by using `w+` access mode.
→ If the file existed in the file location you can perform the reading & writing.

```
fp = open("abc123.txt", "w+")
```

```
fp.write("Welcome to KLU")
```

```
fp.seek(0)
```

```
print(fp.read())
```

```
fp.close()
```

`A+ mode` :-

reading & appending

If the file is available you can perform the both reading and append.

→ If the file doesnot exist in the file location initially to create file then after you can perform the actions.

seek():- It is used to change the current file position within a open file.

→ It allows you to move the (current) cursor to a specific position within the file, to enable the reading a writing modes.

```
fp = open("abc123.txt", "a+")
fp.seek(0)
print(fp.read())
fp.close()
```

readlines:-

```
fp = open("abc123.txt", "r")
print(fp.readlines())
fp.close()
```

Exception Handling:-

To interrupt the program execution that is called Exception. It is also known as a error.

Types of errors:-

- 1) Syntax error
- 2) Logical error
- 3) Runtime error

1) Syntax error:-

If you are missed the semicolon(;), (), { } in your program these errors coming under syntax error.

→ You can easily solve that such type of errors.

Ex:-

```
a=10
b=20
c=a+b
print(c
```

syntax error:-)

2) Logical error:-

a) The flow of execution is successfully completed if you are getting output also but your expectation is different output.

b) It is very difficult to find such type of errors.

ex:-

a=10

b=20

c = ~~10~~ b-a

print(c) #10

expectation

20

3) Runtime error:-

Your program is executed successfully at compilation but you are getting the error at run time, but these runtime error is known as exception.

ex:-

a=10

b=0

c = a/b # Division error

print(c)

You can resolve the runtime errors or exceptions when we are handling 4 types of keywords...

1) try

S-1 - ✓

2) except

S-2 - x

3) else

S-3 - x

4) finally

S-4 - x

1) try:

If S+1 is executed then goes to S-2. If S-2 is executed goes to S-3 otherwise statement will stop at S-2 remaining statements do not executed

Instructions from which we are expecting the exception.

Syntax:- try

S-1

S-2

S-3

2) Except: Exceptions are raised in your try block, the try block will be handled by the except block.

Syntax:- Except:

```
print("error getting")
```

3) else:-

else, finally are optional.

program:-

```
1) a = int(input("Enter A value:"))
```

```
    b = int(input("Enter B value:"))
```

```
    try:
```

```
        c = a/b
```

```
        print(c)
```

```
    except:
```

```
        print("Error is raised")
```

```
2) a = int(input("Enter A value:"))
```

```
    b = int(input("Enter B value:"))
```

```
    try:
```

```
        c = a/b
```

```
        print(c)
```

```
    except:
```

```
        print("Error is raised")
```

```
    else:
```

```
        print("Hello")
```

```
    finally
```

```
        print("KLU")
```

Note

If try is true all statements are executed except except block.

If except stat is true only finally statement block is executed

```
3) a = int(input("Enter A value:"))
```

```
    b = int(input("Enter B value:"))
```

```
    L = [10, 20, 30, 40]
```

```
    try:
```

```
        c = a/b
```

```
        print(c)
```

```

- print(L[5])
- except:
    print("Error is raised")
else:
    print("Welcome to KLD")
finally:
    print("EEE")

```

```

4) a = int(input("Enter A value:"))
b = int(input("Enter B value:"))
L = [10, 20, 30, 40]
try:
    c = a/b
    print(c)
    print(L[5])
except Exception as e:
    print(e)

```

Built-in exception:-

Some built-in exceptions are available in python

- 1) Division by zero
- 2) Index out of range
- 3) Type error
- 3, 4) Value error

* Single exception handling:

In your program the try block maintain the more than one statement, if statement is ^{not} executed depends upon the exception of the except block is executed but the 1st statement is successfully executed sometimes and statement not executed on that time except block is opened but if u are pass the exception in except block or errors.

```
a = int(input("Enter A value:"))
```

```
b = int(input("Enter B value:"))
```

```
L = (10, 20, 30, 40)
```

```
try:
```

```
    c = a/b
```

```
    print(c)
```

```
    print(L[5])
```

```
except ZeroDivisionError:
```

```
    print("Error raised")
```

multiple exception handling:

You are maintaining the more than one except block that is undergoes to the multiple exception.

```
try:
```

```
except 1: . . .
```

```
except 2: . . .
```

```
except 3: . . .
```

```
except (ex built in except 1, built in except 2)
```

If these multiple except blocks you are maintaining into the one single except block by using comma separator.

```
a = int(input())
```

```
b = int(input())
```

```
try L = (10, 20, 30, 40)
```

```
try:
```

```
    c = a/b
```

```
    print(c)
```

```
    print(L[5])
```

- Except zero Division Error:

```
try:  
    print("Error Raised")
```

Except Index Error:

```
print("Index out of Range")
```

By using comma separator:

```
a = int(input())
```

```
b = int(input())
```

```
l = [0, 20, 30, 40]
```

```
try:
```

```
    c = a/b
```

```
    print(c)
```

```
    print(l[5])
```

```
except (ZeroDivisionError, IndexError):
```

```
    print("Error Raised")
```

• Creating a own exceptions:-

```
class InvalidData(Exception):  
    pass
```

```
class InvalidMarks(Exception):  
    pass
```

```
Marks = int(input("Enter a Marks:"))
```

```
try:
```

```
    if (Marks < 0 or Marks > 100):
```

```
        raise InvalidMarks
```

```
    print("Marks =", Marks)
```

```
except:
```

```
    print("Marks should be 0 and 100 only")
```

Use raise keywords

Packages

- 1) The packages are the same as a directory.
- 2) It containing sub dictionaries or sub packages, ^{modules} ~~modules~~ in a structured way.
- 3) You can easily access the sub packages and modules

--init--.py

- > directory that contains the --init--.py file it defined as a python file package.
- > it is an empty file. You can't modify the that python package.
- > A package can be imported the same way as a module is imported.

Structure of Packages

my-package/

--init--.py

Module.

Module1.py

Module2.py

Subpackage1/

--init--.py

Submodule1_1.py

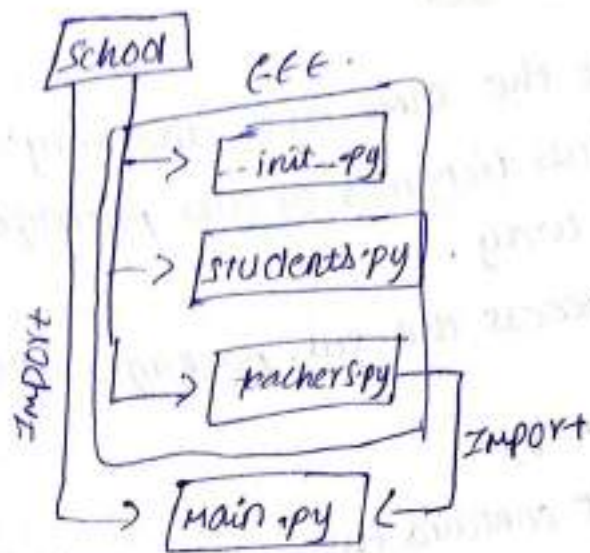
Submodule2_2.py

Subpackage2/

--init--.py

Submodule2_1.py

Submodule2_2.py



Student.py

```
def getstudents():
```

```
    print("Hello")
```

teacher.py

```
def getteacher():
```

```
    print("Hi")
```

Main:

```
from school.students import getstudents
```

```
from school.teacher import getteacher
```

```
getstudents()
```

```
getteacher()
```

* (Interphases) ~~Interphases~~ phases:-

It involves creating a blue print for classes

but that implements certain methods or properties.

→ Python doesnot have explicit interphases faces like

some other programming language like java, c# but

you can achieve the similar function by using abstract

base class by abc module.

→ It involves two types of interfaces.

1) formal interfaces.

2) Informal Interfaces .

Formal Interfaces:-

It's It's play a crucial role in software engineering as they provide to define a set of methods or behaviour or class must be present.

→ As application grows in size & complexity and it becomes more difficult to manage updates & changes to core base.

Informal Interfaces:- It is a class defines methods that can be overridden but there is no restriction and enforcement.

program:-

```
from abc import ABC, abstractmethod  
class shapes(ABC):
```

```
    def area(self):
```

```
        pass
```

```
    def perimeter(self):
```

```
        pass
```

```
class square(shapes):
```

```
    def __init__(self, side):
```

```
        self.side = side
```

```
    def area(self):
```

```
        return self.side * self.side
```

```
class circle(shapes):
```

```
    def __init__(self, radius):
```

```
        self.radius = radius
```

```
    def area(self):
```

```
        return 3.145 * self.radius ** 2
```

```
    def perimeter(self):
```

```
        return 2 * 3.145 * self.radius
```

```
square = square  
print ("area of square =", square.area())
```

```
circle = circle(4)  
print ("area of circle =", circle.area())  
print ("perimeter of circle =", circle.perimeter())
```

Internal:-

```
class Dog:
```

```
    def speak(self):
```

```
        return "boww"
```

```
class cat:
```

```
    def speak(self):
```

```
        return "Meoww"
```

```
class duck:
```

```
    def speak(self):
```

```
        return "Quack"
```

```
def sound(animal):
```

```
    return animal.speak()
```

```
dog = dog()
```

```
cat = cat()
```

```
duck = duck()
```

```
print(sound(dog))
```

```
print(sound(cat))
```

```
print(sound(duck))
```

Difference b/w the ^{phases} interfaces and abstract classes.

Interface

1) It is a set of methods and attributes of the object.

2) All methods of an interface are absent

3) Interfaces is very slow compared to abstract class

abstract.

1) It is used to define & enforce an interface.

2) An abstract class can have abstract methods as well as concrete method.

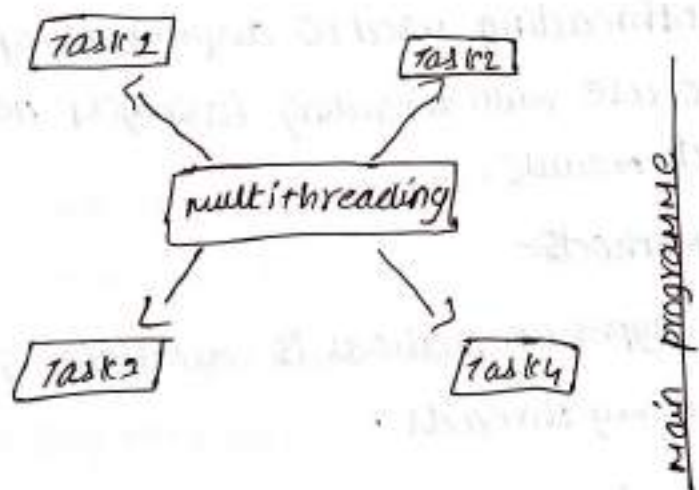
3) Abstract classes are very fast.

Multi threading:-

* A program (or) processes smallest unit is called as thread.

* It can run on its own (or) as part of schedule set by the operating system.

* A computer system is achieved by operating system.



Benefits:-

* It guarantees the powerful usage of PC framework assets.

* The applications with multiple threads and responds very faster.

* It is more cost effective it (shows) shares resources and its state with sub threads.

- By running multiple threads simultaneously it cut down
 $\approx$ of time both threads.
- To store multiple threads the system doesn't require a lot of memory.

How to achieve multithreading:-

These are two main modules of multithreading used to handle the threads.

- thread module
- Threading module.

- Thread module:-

- When we are working on thread module first you
- need to import a thread module and then define a function that performs some action by setting the target with a variable.

1 syntax:-

thread.newthread (target = function name, args = (arg,))

Threading module:-

It is a high level implementation of multithreading used to display an application in python. To use multithreading first you need to import thread module.

3 Thread class Methods:-

3 types of methods is available in thread class to start my thread.

- i) start ():- It is used to initiate the activity of thread. It calls only once for each thread so that the execution of thread can begin.

ii) Join ():- Whenever you are using the join() it is used to block the execution of another code until the thread terminates.

(ii) run() :-

A run() is used to define a thread's activity and can be overridden by a class that extends the thread class.

program:-

```
import time
```

```
def square(numbers):
```

```
    print('square numbers:')
```

```
    for n in numbers:
```

```
        time.sleep(0.7)
```

```
        print("squares:", n*n)
```

```
def cubes(numbers):
```

```
    print("cubes numbers:')
```

```
    for n in numbers:
```

```
        time.sleep(0.5)
```

```
        print("cubes:", n*n*n),
```

```
arr = [2, 4, 6, 8, 10]
```

```
t1 = time.time()
```

```
    square(arr)
```

```
    cubes(arr)
```

```
    print('processing time =', time.time() - t1)
```

Multi threading Program:-

```
import time
```

```
import threading
```

```
def square(numbers):
```

```
    print("square numbers:\n")
```

```
    for n in numbers:
```

```

time.sleep(0.2)
print("squares:", n*n)
def cubes(numbers):
    print("cube numbers:")
    for n in numbers:
        time.sleep(0.4)
        print("cubes:", n*n*n)
arr = [2, 4, 6, 8, 10]
t1 = time.time()
t1 = threading.Thread(target=square,
                      args=(arr,))
t2 = threading.Thread(target=cubes, args=(arr,))
t1.start()
t2.start()
t1.join()
t2.join()

```

Multiprocessing:-

- Every process has its own address base thus the program variables are not shared b/w two processors.
- If you need used to ~~two~~ communicate the IP means interprocess communication chipset technique we want to share two processes.
- multiprocessing can be represented as a computer with more than one central processor.
- A multicore processor refer to single computing component with two or more individual independent processors.

Ex:- `if __name__ == "__main__":`

This block ensure that the code inside it is only executed when the script is run directly, not when its imported as a module.

program:-

```
import multiprocessing
```

```
def square(numbers):
```

```
    for n in numbers:
```

```
        print('square'+str(n*n))
```

```
def cube(numbers):
```

```
    for n in numbers:
```

```
        print('cube'+str(n*n*n))
```

```
if __name__ == "__main__":
```

```
    arr = [2,4,6,8,10]
```

```
    p1 = multiprocessing.Process(target = square,  
                                args = (arr,))
```

```
    p2 = multiprocessing.Process(target = cube,  
                                args = (arr,))
```

```
    p1.start()
```

```
    p2.start()
```

```
    p1.join()
```

```
    p2.join()
```

```
    print("bye")
```

Difference b/w multithreading & multiprocessing

Multi threading

Multi processing

1) It is a technique where multiple threads are generated by a single processor at the same time

2) Python multithreading implements concurrently

3) Multi threads are running simultaneously but working concurrently

1) It is a technique where multiprocessor run across multiple course at the same time

2) Python multi processing implements parallelism.

3) Multi processor run parallelly as the multi processing module streamlines ^{by using} independent and sub processor.